

Guia Practica: API de Transacciones Bancarias

Curso: Desarrollo de Aplicaciones Web

Escuela: Escuela Profesional de Ingenieria de Sistemas e Informatica

IDE: IntelliJ IDEA

Stack: Spring Boot 3.5.14 - Java 17 - Spring Data JPA / Hibernate - PostgreSQL (Supabase) - Maven - Postman

Patron arquitectonico: CONTROLLER -> SERVICE -> REPOSITORY -> BASE DE DATOS

1. Que vamos a construir

Una API REST con CRUD completo para gestionar **transacciones bancarias** (debito y credito) asociadas a un usuario y a una aplicacion expone 6 endpoints HTTP que se prueban con Postman, persiste los datos en PostgreSQL alojado en Supabase, y a negocio (validacion de tipo, monto y campos obligatorios).

Flujo de una peticion HTTP

```
POST /api/transacciones    <- el cliente (Postman) envia el request
    |
    v
@PostMapping              <- ROUTER (anotacion del Controller)
    |
    v
TransaccionController.java  <- CONTROLLER: recibe HTTP, devuelve JSON
    |
    v
TransaccionServiceImpl.java <- SERVICE: aplica reglas de negocio
    |
    v
TransaccionRepository.java  <- REPOSITORY: habla con la BD via JPA
    |
    v
Tabla "transacciones"      <- PostgreSQL en Supabase
```

Por que separar en capas

Cada capa tiene **una sola responsabilidad**.

- Si mañana cambias PostgreSQL por MySQL, solo tocas la configuracion del Repository.
- Si mañana piden aceptar un nuevo tipo de transaccion ("transferencia"), solo tocas el Service.
- El Controller no sabe ni de reglas ni de SQL: solo de HTTP (rutas, codigos de estado, JSON).

Esta separacion es lo que el curso llama "arquitectura por capas" y es el estandar industrial para APIs REST en Java.

2. Requisitos previos

2.1. Java 17 y Maven

```
java -version
# openjdk version "17.x.x"

mvn -version
# Apache Maven 3.x.x
```

Si no los tienes, instala el JDK 17 desde adoptium.net y Maven desde maven.apache.org. En Windows se recomienda agregar las variables de entorno.

2.2. Cuenta de Supabase

1. Crear un proyecto gratuito en supabase.com.
2. Ir a **Project Settings** -> **Database** -> **Connection string** y elegir el modo **Session pooler** (compatible con IPv4 y con el c
3. Anotar:
 - Host (ejemplo: `aws-1-us-east-2.pooler.supabase.com`)
 - Puerto: 5432
 - Database: postgres
 - Username: `postgres.<id-del-proyecto>`
 - Password: el que definiste al crear el proyecto.

3. Estructura final del proyecto

```

transacciones/
├── pom.xml                                <- dependencias Maven
├── src/
│   └── main/
│       ├── java/com/banco/transacciones/
│       │   ├── TransaccionesApplication.java    <- punto de entrada
│       │   ├── controllers/
│       │   │   └── TransaccionController.java   <- CAPA CONTROLLER
│       │   ├── services/
│       │   │   ├── TransaccionService.java      <- INTERFAZ del Service
│       │   │   └── TransaccionServiceImpl.java  <- IMPLEMENTACION del Service
│       │   ├── repositories/
│       │   │   └── TransaccionRepository.java    <- CAPA REPOSITORY
│       │   ├── models/
│       │   │   └── Transaccion.java              <- ENTIDAD JPA
│       │   ├── dto/
│       │   │   └── TransaccionRequestDto.java    <- DTO de entrada
│       └── resources/
│           └── application.properties            <- configuracion
└── target/                                     <- compilado por Maven
  
```

4. Crear el proyecto desde IntelliJ IDEA

El proyecto se crea directamente desde IntelliJ IDEA usando el asistente integrado de **Spring Boot** (que internamente consume Spring Initializr, pero sin necesidad de visitar `start.spring.io` ni descargar archivos ZIP).

4.1. Pasos en el asistente New Project

1. Abre IntelliJ IDEA y haz clic en **New Project** (en la pantalla de bienvenida) o **File -> New -> Project...** si ya tienes otro pro
2. En el panel izquierdo, selecciona **Spring Boot** (en algunas versiones aparece como **Spring Initializr**).
3. Completa los campos del primer paso:

Campo	Valor
Name	transacciones
Location	D:\GuillEdu\BackEnd\SprintBoot\s2_m3_transacciones_spring_supabase
Language	Java
Type	Maven
Group	com.banco
Artifact	transacciones
Package name	com.banco.transacciones
JDK	17 (Oracle OpenJDK / Temurin / Adoptium)

Campo	Valor
Java	17
Packaging	Jar

4. Haz clic en **Next**.

4.2. Seleccionar dependencias

En el segundo paso del asistente IntelliJ muestra el catalogo de dependencias de Spring Initializr. Marca las siguientes:

- **Spring Web** (categoria *Web*) - controladores REST y Tomcat embebido.
- **Spring Data JPA** (categoria *SQL*) - JPA + Hibernate.
- **PostgreSQL Driver** (categoria *SQL*) - driver JDBC para Supabase.
- **Spring Boot DevTools** (categoria *Developer Tools*) - recarga en caliente.
- **Lombok** (categoria *Developer Tools*) - getters/setters automaticos (opcional).

Confirma la version de Spring Boot (3.5.x) y haz clic en **Create**.

4.3. Que pasa despues

IntelliJ realiza automaticamente lo siguiente:

1. Crea la estructura de carpetas (`src/main/java` , `src/main/resources` , `src/test/java`).
2. Genera el `pom.xml` con todas las dependencias seleccionadas.
3. Crea la clase principal `TransaccionesApplication.java` con la anotacion `@SpringBootApplication` .
4. Descarga las dependencias Maven (veras el progreso en la barra inferior, en la pestania **Build**).
5. Indexa el proyecto para que el autocompletado funcione.

No es necesario abrir terminal, ni descomprimir ZIP, ni ejecutar comandos `mvn` manualmente. El asistente de IntelliJ se todo el ciclo de creacion.

A partir de aqui ya puedes empezar a crear los paquetes (`controllers` , `services` , `repositories` , `models` , `dto`) usando clic der `com.banco.transacciones` -> `New` -> `Package` .

5. Dependencias - `pom.xml`

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.14</version>
    <relativePath/>
  </parent>

  <groupId>com.banco</groupId>
  <artifactId>transacciones</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>transacciones</name>

  <properties>
    <java.version>17</java.version>
  </properties>
```

Dependencia	Para que sirve
spring-boot-starter-web	Habilita Spring MVC, Tomcat embebido y la creacion de controladores REST
spring-boot-starter-data-jpa	Trae JPA + Hibernate y la magia de los repositorios automaticos
postgresql	Driver JDBC para conectarse a PostgreSQL/Supabase
spring-boot-devtools	Recarga automatica al guardar cambios (live reload)
lombok	Genera getters/setters/construtores con anotaciones (opcional)
spring-boot-starter-test	JUnit + Mockito para pruebas unitarias

Para descargar las dependencias:

```
mvn clean install
```

6. Configuracion - src/main/resources/application.properties

```
# ===== Servidor =====
server.port=8080
spring.application.name=transacciones

# ===== Conexion a Supabase PostgreSQL (Session Pooler / IPv4) =====
spring.datasource.url=jdbc:postgresql://aws-1-us-east-2.pooler.supabase.com:5432/postgres
spring.datasource.username=postgres.<tu-id-de-proyecto>
spring.datasource.password=<tu-password>
spring.datasource.driver-class-name=org.postgresql.Driver

# ===== HikariCP (pool de conexiones) =====
spring.datasource.hikari.maximum-pool-size=5
spring.datasource.hikari.connection-timeout=30000

# ===== JPA / Hibernate =====
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Explicacion linea por linea:

- `server.port=8080` : el Tomcat embebido escuchara en `http://localhost:8080` .
- `spring.datasource.url` : cadena JDBC. Usamos el **Session Pooler** de Supabase porque es compatible con redes IPv4 (en W hogareno casi siempre lo es).
- `spring.datasource.username` / `password` : credenciales del usuario `postgres` del pooler. **No subas el password real a Git**; us entorno o `.env` en proyectos serios.
- `spring.jpa.hibernate.ddl-auto=update` : Hibernate compara tus entidades con la BD y crea/altera tablas automaticamente. l desarrollo, peligroso en produccion (alli se usa `validate` + Flyway/Liquibase).
- `spring.jpa.show-sql=true` y `format_sql=true` : imprimen el SQL generado en la consola, formateado. Excelente para aprend

7. Punto de entrada - TransaccionesApplication.java

```
package com.banco.transacciones;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class TransaccionesApplication {

    public static void main(String[] args) {
        SpringApplication.run(TransaccionesApplication.class, args);
    }
}
```

```
}  
}
```

Que hace @SpringBootApplication :

Es una anotacion combinada que activa tres comportamientos:

1. @Configuration : declara que la clase puede contener beans (componentes gestionados por Spring).
2. @EnableAutoConfiguration : Spring detecta las dependencias del classpath y configura todo automaticamente (Tomcat, JPA, etc.).
3. @ComponentScan : escanea el paquete actual (com.banco.transacciones) y todos sus subpaquetes buscando clases anotadas con @Controller , @Service , @Repository , @Component .

SpringApplication.run(...) levanta el contexto de Spring, arranca Tomcat embebido y deja la app escuchando en el puerto 8080

8. Capa MODEL - Entidad Transaccion.java

La entidad es la **representacion de una fila** de la tabla transacciones . JPA/Hibernate la usa para mapear objetos Java a registros de la BD

```
package com.banco.transacciones.models;  
  
import jakarta.persistence.*;  
import java.math.BigDecimal;  
import java.time.LocalDateTime;  
import java.util.UUID;  
  
@Entity  
@Table(name = "transacciones")  
public class Transaccion {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.UUID)  
    private UUID id;  
  
    @Column(name = "user_id", nullable = false)  
    private UUID userId;  
  
    @Column(name = "cuenta_id", nullable = false)  
    private UUID cuentaId;  
  
    @Column(nullable = false)  
    private String tipo;
```

Anotaciones explicadas:

Anotacion	Significado
@Entity	Le dice a JPA: "esta clase es una tabla".
@Table(name = "transacciones")	Nombre exacto de la tabla en la BD (sin esto, usaria Transaccion).
@Id	Marca el campo como clave primaria.
@GeneratedValue(strategy = GenerationType.UUID)	Hibernate genera un UUID automaticamente al insertar.
@Column(name = "user_id")	Mapea userId (camelCase Java) a user_id (snake_case SQL). Esto es clave porque PostgreSQL es case-sensitive con identificadores.
nullable = false	Equivale a NOT NULL en SQL.
precision = 12, scale = 2	Para BigDecimal : hasta 12 digitos en total, 2 decimales (ejemplo: 9999999999.99).

Por que BigDecimal y no double : los double pierden precision en operaciones financieras (0.1 + 0.2 = 0.30000000000000004).
exacto y obligatorio para dinero.

Por que UUID y no Long : los UUID son globalmente unicos, no revelan cuantos registros hay y son ideales para sistemas distribuidos. Supabase los maneja nativamente con el tipo `uuid`.

9. Capa DTO - TransaccionRequestDto.java

Un **DTO (Data Transfer Object)** es un objeto que viaja entre el cliente y el servidor. Sirve para **separar la forma del request de la entidad**.

```
package com.banco.transacciones.dto;

import java.math.BigDecimal;
import java.util.UUID;

public class TransaccionRequestDto {

    private UUID userId;
    private UUID cuentaId;
    private String tipo;
    private String descripcion;
    private BigDecimal monto;

    public UUID getUserId() { return userId; }
    public void setUserId(UUID userId) { this.userId = userId; }

    public UUID getCuentaId() { return cuentaId; }
    public void setCuentaId(UUID cuentaId) { this.cuentaId = cuentaId; }

    public String getTipo() { return tipo; }
    public void setTipo(String tipo) { this.tipo = tipo; }

    public String getDescripcion() { return descripcion; }
```

Por que necesitamos un DTO si ya existe la entidad Transaccion :

1. **Seguridad:** el cliente NO debe poder mandar el `id` ni la `fecha` (los genera el servidor). El DTO solo expone los campos que necesitamos.
2. **Independencia:** si mañana renombras la columna `descripcion` a `detalle` en la BD, el DTO sigue siendo `descripcion` y la API no se rompe.
3. **Validaciones distintas:** la entidad puede tener `nullable = false` para SQL, pero el DTO puede tener reglas de negocio mas complejas (ejemplo: longitud minima del texto).
4. **Claridad:** cuando alguien lee `registrarTransaccion(TransaccionRequestDto dto)` ve exactamente que datos espera la API.

Regla didactica: la **entidad** es como debe verse el dato en la base de datos; el **DTO** es como debe verse el dato en la peticion.

10. Capa REPOSITORY - TransaccionRepository.java

El Repository es la **unica capa que toca la base de datos**. En Spring Data JPA basta con declarar una interfaz que extiende `JpaRepository` y Spring genera la implementacion automaticamente en tiempo de ejecucion.

```
package com.banco.transacciones.repositories;

import com.banco.transacciones.models.Transaccion;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.UUID;

@Repository
public interface TransaccionRepository extends JpaRepository<Transaccion, UUID> {

    List<Transaccion> findByCuentaIdOrderByFechaDesc(UUID cuentaId);
```

```
List<Transaccion> findByUserIdOrderByFechaDesc(UUID userId);
}
```

Que heredas gratis de `JpaRepository<Transaccion, UUID>` :

Metodo heredado	Que hace
<code>save(Transaccion t)</code>	INSERT (si no existe) o UPDATE (si tiene id)
<code>findById(UUID id)</code>	SELECT por clave primaria (devuelve <code>Optional<Transaccion></code>)
<code>findAll()</code>	SELECT *
<code>existsById(UUID id)</code>	Devuelve <code>true</code> si existe
<code>deleteById(UUID id)</code>	DELETE por id
<code>count()</code>	COUNT(*)

Magia de los nombres de metodos (Query Derivation):

Cuando declaras `findByCuentaIdOrderByFechaDesc` , Spring lee el nombre del metodo y genera el SQL:

```
SELECT * FROM transacciones
WHERE cuenta_id = ?
ORDER BY fecha DESC;
```

Las palabras clave que reconoce: `findBy` , `And` , `Or` , `OrderBy` , `Asc` , `Desc` , `Like` , `Between` , `GreaterThan` , etc. No tienes que escribir JPQL.

`@Repository` marca la interfaz como un componente de acceso a datos. Spring la detecta en el escaneo y la inyecta donde se

11. Capa SERVICE - Reglas de negocio

Esta capa contiene las **reglas del negocio bancario**: que tipos de transaccion son validos, que montos se aceptan, que campos obligatorios. Se separa en dos archivos: la **interfaz** (contrato) y la **implementacion** (logica).

11.1. Interfaz - TransaccionService.java

```
package com.banco.transacciones.services;

import com.banco.transacciones.dto.TransaccionRequestDto;
import com.banco.transacciones.models.Transaccion;

import java.util.List;
import java.util.UUID;

public interface TransaccionService {

    List<Transaccion> listarPorCuenta(UUID cuentaId);

    List<Transaccion> listarPorUsuario(UUID userId);

    Transaccion registrarTransaccion(TransaccionRequestDto dto);

    Transaccion actualizarTransaccion(UUID id, TransaccionRequestDto dto);

    void eliminarTransaccion(UUID id);
}
```

11.2. Implementacion - TransaccionServiceImpl.java

```
package com.banco.transacciones.services;

import com.banco.transacciones.dto.TransaccionRequestDto;
```

```
import com.banco.transacciones.models.Transaccion;
import com.banco.transacciones.repositories.TransaccionRepository;
import org.springframework.stereotype.Service;

import java.math.BigDecimal;
import java.util.Arrays;
import java.util.List;
import java.util.UUID;

@Service
public class TransaccionServiceImpl implements TransaccionService {

    private final TransaccionRepository transaccionRepo;

    public TransaccionServiceImpl(TransaccionRepository transaccionRepo) {
        this.transaccionRepo = transaccionRepo;
    }

    @Override
    public void crearTransaccion(String tipo, double monto, String descripcion,
                                String userId, String cuentaId) {
```

Reglas de negocio que viven aqui:

1. **Tipo valido:** solo "debito" o "credito" . Cualquier otro valor lanza error.
2. **Monto positivo:** debe ser mayor a 0 (no acepta 0 ni negativos).
3. **Descripcion obligatoria:** no puede ser null ni cadena vacia ni solo espacios.
4. **Identificadores obligatorios:** userId y cuentaId no pueden ser nulos.
5. **PUT parcial:** en la actualizacion, los campos nulos se ignoran (no se sobrescribe lo que no se envia). Esto es un PATCH de PUT, muy comun en APIs reales.
6. **DELETE seguro:** verifica primero existsById para devolver un error claro si el id no existe.

Por que separar interfaz e implementacion:

- La inyeccion de dependencias permite cambiar la implementacion (ej. un TransaccionServiceMock para tests) sin tocar el C
- Es el principio de **Inversion de Dependencias** (la D de SOLID).

Inyeccion por constructor (recomendada):

```
public TransaccionServiceImpl(TransaccionRepository transaccionRepo) {
    this.transaccionRepo = transaccionRepo;
}
```

Spring detecta el constructor y le pasa automaticamente una instancia de TransaccionRepository . Es preferible a @Autowired e porque:

- Permite declarar final (inmutabilidad).
- Facilita los tests (puedes pasar mocks directamente).
- Falla en compilacion si te falta una dependencia.

@Service marca la clase como un componente de logica de negocio. Funcionalmente equivale a @Component , pero comunica i lector.

12. Capa CONTROLLER - TransaccionController.java

El Controller es la **puerta de entrada HTTP**. Su unica responsabilidad es:

1. Leer la peticion (URL, query params, body, path variables).
2. Llamar al Service.
3. Devolver la respuesta como JSON con el codigo HTTP correcto.

No debe contener if de reglas de negocio ni consultas SQL.

```

package com.banco.transacciones.controllers;

import com.banco.transacciones.dto.TransaccionRequestDto;
import com.banco.transacciones.models.Transaccion;
import com.banco.transacciones.services.TransaccionService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.UUID;

@RestController
@RequestMapping("/api/transacciones")
@CrossOrigin(origins = "**")
public class TransaccionController {

    private final TransaccionService transaccionService;

    public TransaccionController(TransaccionService transaccionService) {
        this.transaccionService = transaccionService;
    }
}

```

Anotaciones explicadas:

Anotacion	Funcion
@RestController	Combina @Controller + @ResponseBody . Cada metodo devuelve JSON automaticamente.
@RequestMapping("/api/transacciones")	Prefijo comun de todas las rutas de la clase.
@CrossOrigin(origins = "**")	Habilita CORS desde cualquier origen. Util en desarrollo cuando un frontend (React/Angular/Vue) corre en otro servidor.
@GetMapping / @PostMapping / @PutMapping / @DeleteMapping	Atajos para @RequestMapping(method = ...) .
@RequestParam UUID cuentaId	Lee ?cuentaId=... del query string.
@PathVariable UUID id	Lee la variable de la URL: /api/transacciones/{id} .
@RequestBody TransaccionRequestDto	Convierte el JSON del body a un objeto Java (lo hace Jackson automaticamente).
ResponseEntity<?>	Permite controlar el codigo HTTP (200, 201, 400, 404...) y el cuerpo de respuesta.

Patron de respuesta uniforme:

Todas las respuestas siguen el formato:

Cuando todo sale bien:

```
{
  "success": true,
  "data": {}
}
```

Cuando hay un error de validacion:

```
{
  "success": false,
  "message": "Tipo invalido. Debe ser 'debito' o 'credito'"
}
```

Esto facilita al frontend manejar las respuestas con un solo `if (res.success)`.

13. Inyeccion de dependencias en Spring

A diferencia de ASP.NET (donde tienes que registrar manualmente cada servicio en `Program.cs`), en Spring **el escaneo de con auto-configuracion lo hacen todo solo**. Tu solo anotas las clases:

Anotacion	Donde
@Repository	En la interfaz que extiende <code>JpaRepository</code>
@Service	En la implementacion del Service
@RestController	En el Controller

Y cuando declaras un constructor que pide una dependencia, Spring la inyecta:

@RestController	pide	TransaccionService	-> Spring le pasa	TransaccionServiceImpl
@Service	pide	TransaccionRepository	-> Spring le pasa	proxy generado por JPA
TransaccionRepository	pide	EntityManager	-> Spring le pasa	uno gestionado
EntityManager	pide	DataSource	-> Spring le pasa	HikariCP -> PostgreSQL

Este "arbol" se construye automaticamente al arrancar la aplicacion, basado en `application.properties` y las anotaciones.

14. Ejecutar la API

14.1. Desde la terminal (Maven)

```
mvn spring-boot:run
```

14.2. Desde IntelliJ IDEA

Hay tres formas:

1. **Boton verde junto al main** : abre `TransaccionesApplication.java` , IntelliJ muestra una flecha verde (Play) en la columna izq `public static void main(...)` . Haz clic y selecciona **Run 'TransaccionesApplication'**.
2. **Atajo de teclado**: con la clase abierta, presiona `Shift + F10` para correr la ultima configuracion, o `Ctrl + Shift + F10` para una nueva.
3. **Maven Tool Window**: abre la pestania **Maven** en el panel derecho y ejecuta el goal `Plugins -> spring-boot -> spring-boot`

Para detener la aplicacion: clic en el cuadrado rojo (Stop) de la ventana **Run** o presiona `Ctrl + F2`.

14.3. Salida esperada en consola

[illegible]

```
'  ____| .__/_| |__| |_\__, | / / / /
=====|_|=====|_|_/=_/_/_/_/
```

```
:: Spring Boot ::                (v3.5.14)
```

```
INFO ... Started TransaccionesApplication in 4.567 seconds
INFO ... Tomcat started on port 8080
```

Si ves esto, la API esta lista en `http://localhost:8080/api/transacciones` .

14.4. Live reload con DevTools

Al guardar un archivo `.java` , Spring DevTools reinicia el contexto en 1-2 segundos. No hace falta detener y volver a correr.

15. Endpoints disponibles (resumen)

Base URL: `http://localhost:8080/api/transacciones`

#	Metodo	Ruta	Descripcion	Codi
1	POST	<code>/api/transacciones</code>	Registrar una transaccion (debito o credito)	201
2	GET	<code>/api/transacciones?cuentaId={uuid}</code>	Listar transacciones de una cuenta (mas recientes primero)	200
3	GET	<code>/api/transacciones/usuario/{userId}</code>	Listar transacciones de un usuario	200
4	PUT	<code>/api/transacciones/{id}</code>	Actualizar una transaccion (parcial)	200
5	DELETE	<code>/api/transacciones/{id}</code>	Eliminar una transaccion	200

16. Pruebas con Postman

16.1. Endpoint 1 - POST debito

Request:

```
POST http://localhost:8080/api/transacciones
Content-Type: application/json
```

Body (raw, JSON):

```
{
  "userId": "f03363d1-c24d-463b-a654-4c38dd590007",
  "cuentaId": "429efe8d-93c7-4b9b-a58b-74d4630f86a8",
  "tipo": "debito",
  "descripcion": "Pago internet Movistar",
  "monto": 89.90
}
```

Respuesta esperada (201 Created):

```
{
  "success": true,
  "data": {
    "id": "8e359251-009b-45a2-89ab-29769a1479db",
    "userId": "f03363d1-c24d-463b-a654-4c38dd590007",
    "cuentaId": "429efe8d-93c7-4b9b-a58b-74d4630f86a8",
    "tipo": "debito",
    "descripcion": "Pago internet Movistar",
    "monto": 89.90,
    "fecha": "2026-04-27T10:15:30.123"
  }
}
```

Anota el `id` que devuelve la respuesta: lo usaras para los endpoints PUT y DELETE.

16.2. Endpoint 2 - POST credito

Request:

POST `http://localhost:8080/api/transacciones`
Content-Type: `application/json`

Body:

```
{
  "userId": "f03363d1-c24d-463b-a654-4c38dd590007",
  "cuentaId": "429efe8d-93c7-4b9b-a58b-74d4630f86a8",
  "tipo": "credito",
  "descripcion": "Bono trabajo",
  "monto": 1200.00
}
```

Respuesta esperada (201 Created):

```
{
  "success": true,
  "data": {
    "id": "<uuid-generado>",
    "userId": "f03363d1-c24d-463b-a654-4c38dd590007",
    "cuentaId": "429efe8d-93c7-4b9b-a58b-74d4630f86a8",
    "tipo": "credito",
    "descripcion": "Bono trabajo",
    "monto": 1200.00,
    "fecha": "2026-04-27T10:16:01.456"
  }
}
```

16.3. Endpoint 3 - GET listar por cuenta

Request:

GET `http://localhost:8080/api/transacciones?cuentaId=429efe8d-93c7-4b9b-a58b-74d4630f86a8`

Respuesta esperada (200 OK):

```
{
  "success": true,
  "data": [
    {
      "id": "...",
      "userId": "f03363d1-c24d-463b-a654-4c38dd590007",
      "cuentaId": "429efe8d-93c7-4b9b-a58b-74d4630f86a8",
      "tipo": "credito",
      "descripcion": "Bono trabajo",
      "monto": 1200.00,
      "fecha": "2026-04-27T10:16:01.456"
    },
    {
      "id": "8e359251-009b-45a2-89ab-29769a1479db",
      "userId": "f03363d1-c24d-463b-a654-4c38dd590007",
      "cuentaId": "429efe8d-93c7-4b9b-a58b-74d4630f86a8",
      "tipo": "debito",
      "descripcion": "Pago internet Movistar",
      "monto": 89.90,
      "fecha": "2026-04-27T10:15:30.123"
    }
  ]
}
```

Las transacciones vienen **ordenadas de la mas reciente a la mas antigua** gracias al metodo `findByCuentaIdOrderByFechaDesc`

16.4. Endpoint 4 - GET listar por usuario

Request:

GET `http://localhost:8080/api/transacciones/usuario/6c6cc52f-98a3-4d2d-b98f-f074fef9b699`

Respuesta esperada (200 OK):

```
{
  "success": true,
  "data": []
}
```

Si el usuario no tiene transacciones, devuelve un arreglo vacio. Cambia el UUID por uno que si tenga registros (ejemplo: `f074fef9b699-463b-a654-4c38dd590007` de los POST anteriores) para ver datos.

16.5. Endpoint 5 - PUT actualizar transaccion

Request:

PUT `http://localhost:8080/api/transacciones/8e359251-009b-45a2-89ab-29769a1479db`
Content-Type: `application/json`

Body:

```
{
  "descripcion": "Deposito gratificacion",
  "monto": 10000.00
}
```

Respuesta esperada (200 OK):

```
{
  "success": true,
  "data": {
    "id": "8e359251-009b-45a2-89ab-29769a1479db",
    "userId": "f03363d1-c24d-463b-a654-4c38dd590007",
    "cuentaId": "429efe8d-93c7-4b9b-a58b-74d4630f86a8",
    "tipo": "debito",
    "descripcion": "Deposito gratificacion",
    "monto": 10000.00,
    "fecha": "2026-04-27T10:15:30.123"
  }
}
```

El PUT es **parcial**: solo actualiza los campos que envias. `userId`, `cuentaId`, `tipo` y `fecha` se preservan del registro original.

16.6. Endpoint 6 - DELETE eliminar transaccion

Request:

DELETE `http://localhost:8080/api/transacciones/bb90398c-963c-4b56-bfd3-eb48d65a9be4`

Respuesta esperada (200 OK):

```
{
  "success": true,
}
```

```
"message": "Transaccion eliminada correctamente"
}
```

Si el id no existe (400 Bad Request):

```
{
  "success": false,
  "message": "Transaccion no encontrada con id: bb90398c-963c-4b56-bfd3-eb48d65a9be4"
}
```

17. Flujo sugerido de prueba en Postman

Sigue este orden para validar todo el CRUD:

1. **POST debito** -> copia el `id` que devuelve.
2. **POST credito** -> copia el `id`.
3. **GET por cuenta** -> verifica que aparecen las dos transacciones.
4. **GET por usuario** -> verifica que aparecen filtradas por usuario.
5. **PUT** -> usa el `id` del POST debito; cambia descripcion y monto.
6. **GET por cuenta** -> verifica los cambios.
7. **DELETE** -> elimina con el `id` del POST credito.
8. **GET por cuenta** -> ya no debe aparecer la eliminada.

Pruebas de validacion (deben devolver 400 Bad Request)

Sirven para demostrar que las reglas de negocio del Service funcionan:

Caso	JSON	Mensaje esperado
Tipo invalido	<code>{"tipo": "transferencia", ...}</code>	"Tipo invalido. Debe ser 'debito' o 'credito'"
Monto cero	<code>{"monto": 0, ...}</code>	"Monto debe ser mayor a 0"
Monto negativo	<code>{"monto": -50, ...}</code>	"Monto debe ser mayor a 0"
Sin descripcion	<code>{"descripcion": "", ...}</code>	"Descripcion es requerida"
Sin userId	<code>{"userId": null, ...}</code>	"userId y cuentaId son requeridos"

18. Verificar la tabla en Supabase

Como `spring.jpa.hibernate.ddl-auto=update` esta activado, al arrancar la aplicacion Hibernate crea la tabla automaticamente. F

1. Entra al panel de Supabase -> **Table Editor**.
2. Deberias ver la tabla `transacciones` con las columnas:

Columna	Tipo
<code>id</code>	uuid (PK)
<code>user_id</code>	uuid (NOT NULL)
<code>cuenta_id</code>	uuid (NOT NULL)
<code>tipo</code>	varchar (NOT NULL)
<code>descripcion</code>	varchar (NOT NULL)
<code>monto</code>	numeric(12,2) (NOT NULL)
<code>fecha</code>	timestamp (NOT NULL)

3. Tambien puedes ejecutar en el **SQL Editor**:

```
SELECT * FROM transacciones ORDER BY fecha DESC;
```

19. Comparacion con otros frameworks

Si vienes de Laravel, Node/Express o ASP.NET Core, esta tabla traduce conceptos:

Concepto	Laravel	Node/Express	ASP.NET Core	Spring Boot
Crear proyecto	composer create-project	npm init	dotnet new webapi	Spring Initializr
Instalar dependencia	composer require X	npm install X	dotnet add package X	editar pom.xml + mvn install
Correr app	php artisan serve	node server.js	dotnet run	mvn spring-boot:run
ORM	Eloquent	Prisma / Sequelize	Entity Framework	Spring Data JPA / Hibernate
Migraciones	php artisan migrate	prisma migrate	dotnet ef database update	ddl-auto=update (dev) o Flyway (prod)
Controlador REST	Route::post(...)	app.post(...)	[HttpPost]	@PostMapping
Inyeccion de dep.	Service Container	Manual	builder.Services.Add...	@Component + escaneo

20. Tabla resumen de capas

Capa	Archivo	Responsabilidad	Que NO debe hacer
CONTROLLER	TransaccionController.java	Mapear URLs, leer JSON, devolver JSON, codigos HTTP	Reglas de negocio, logica
SERVICE (interfaz)	TransaccionService.java	Definir el contrato publico de la logica	Implementar nada
SERVICE (impl)	TransaccionServiceImpl.java	Validar reglas, orquestar logica	Tocar HTTP, ejecutar consultas
REPOSITORY	TransaccionRepository.java	Consultas a la BD via JPA	Validar reglas de negocio
MODEL / Entidad	Transaccion.java	Estructura de datos mapeada a la tabla	Logica, validaciones
DTO	TransaccionRequestDto.java	Forma del JSON de entrada	Persistencia, logica
CONFIG	application.properties	Conexion BD, puerto, JPA	Codigo
ENTRY	TransaccionesApplication.java	Arrancar el contexto Spring	Logica

21. Glosario rapido

- **Bean**: objeto gestionado por Spring (creado, inyectado y destruido por el contenedor).
- **Contexto de Spring (ApplicationContext)**: el "almacen" donde viven todos los beans.
- **Inversion de Control (IoC)**: en lugar de que tu crees objetos con `new`, los pides y Spring te los entrega.
- **Inyeccion de Dependencias (DI)**: forma concreta de IoC, normalmente via constructor.
- **JPA (Jakarta Persistence API)**: especificacion estandar de Java para ORM.
- **Hibernate**: implementacion mas usada de JPA. Es la que trae Spring Data JPA por defecto.
- **DTO**: objeto que viaja por HTTP, separado de la entidad para no exponer la BD.
- **CRUD**: Create, Read, Update, Delete - las cuatro operaciones basicas sobre datos.
- **REST**: estilo arquitectonico donde URLs representan recursos y verbos HTTP las acciones.

22. Para profundizar

Una vez funcionando este CRUD basico, los proximos temas a explorar son:

1. **Validacion declarativa** con `jakarta.validation` (`@NotNull` , `@NotBlank` , `@Positive`) en el DTO.
2. **Manejo global de excepciones** con `@RestControllerAdvice` para no repetir try/catch en cada metodo del Controller.
3. **DTO de respuesta** (`TransaccionResponseDto`) para no exponer la entidad directamente.
4. **MapStruct** o **ModelMapper** para mapear DTO <-> Entidad sin escribir setters a mano.
5. **Spring Security + JWT** para autenticar las peticiones.
6. **Tests unitarios** con JUnit 5 + Mockito sobre el Service.
7. **Migraciones formales** con Flyway o Liquibase en lugar de `ddl-auto=update` .
8. **OpenAPI / Swagger UI** con `springdoc-openapi` para documentar la API automaticamente.

Documento generado para el curso de Desarrollo de Aplicaciones Web
Escuela Profesional de Ingenieria de Sistemas e Informatica